IGES

Luciano Bercini

March 2026

Contents

1 Context of the Project **2**

2 Goals of the Project **2**

3 Analysis through Reverse Engineering **3**
3.1 Architectural Overview . . . 3
3.2 Package Diagram . . . 3
3.3 Dependency Analysis . . . 4
3.4 Class Diagram . . . 5
3.5 Sequence Diagram of the Current Workflow . . . 6
3.6 Analysis of External Components . . . 6
3.6.1 SCsVulLyzer . . . 7
3.6.2 Machine Learning Models . . . 7
3.7 Sequence Diagram of the Proposed Workflow . . . 7
3.8 Summary of Findings . . . 8

4 Proposed Changes & Contributions **8**
4.1 CR_1: Integration of Feature Extraction Module . . . 8
4.1.1 Methodology . . . 8
4.1.2 Expected Results . . . 9
4.2 CR_2: Machine Learning-Based Tool Selection . . . 9
4.2.1 Methodology . . . 9
4.2.2 Expected Results . . . 9
4.3 CR_3: Integration into SmartBugs Workflow . . . 9
4.3.1 Methodology . . . 9
4.3.2 Expected Results . . . 10

5 Impact Analysis **10**
5.1 CR_1: Integration of Feature Extraction Module . . . 10
5.2 CR_2: Machine Learning-Based Tool Selection . . . 10
5.3 CR_3: Integration into SmartBugs Workflow . . . 11
5.4 Overall Impact . . . 11

6 Testing Strategy and Assessment **12**
6.1 Testing Strategy . . . 12
6.2 Adequacy Criteria . . . 12
6.3 Testing Tools . . . 12
6.4 Regression Testing Approach . . . 13
6.5 Assessment of the Existing Test Suite . . . 13

1 Context of the Project

The project focuses on the evolution of *SmartBugs*, an extensible framework designed for the automated analysis of Ethereum smart contracts. SmartBugs provides a unified interface to execute multiple vulnerability detection tools, such as static and dynamic analyzers, by encapsulating them within Docker containers.

The framework allows users to:

- Execute multiple analysis tools on one or more smart contracts
- Standardize the output of heterogeneous tools
- Automatically manage Solidity compiler versions

Currently, SmartBugs supports two main execution modes:

- Execution of all available tools
- Execution of a user-specified subset of tools

Despite its flexibility, SmartBugs lacks an intelligent mechanism to select the most appropriate tools based on the characteristics of the input smart contract. This often leads to inefficient analyses, unnecessary computational overhead, and redundant tool executions.

To address this limitation, this project proposes an extension of SmartBugs by integrating:

- A feature extraction tool (*SCsVulLyzer*)
- A set of pre-trained Machine Learning models

The goal is to enable an *intelligent tool selection mechanism* that automatically determines which analysis tools should be executed for a given smart contract.

2 Goals of the Project

The main goal of this project is to extend SmartBugs by introducing a new execution mode capable of automatically selecting the most suitable vulnerability detection tools based on the structural characteristics of a smart contract.

More specifically, the project aims to:

- Integrate *SCsVulLyzer* to extract features from Solidity smart contracts
- Load and execute a set of pre-trained Machine Learning models (stored as `.joblib` files)
- Use these models to predict the most appropriate analysis tools for different vulnerability categories
- Aggregate the predictions and automatically construct the list of tools to execute
- Invoke SmartBugs using the selected tools without modifying its core execution pipeline

The proposed solution introduces a new **intelligent command** that follows this workflow:

1. Extract features from the input smart contract
2. Select the relevant subset of features required by the models
3. Execute multiple ML models (one per vulnerability category)
4. Collect predicted tools
5. Filter results (remove duplicates and irrelevant outputs)
6. Execute SmartBugs with the selected tools

A key design goal is to ensure that the modification:

- Is modular and non-intrusive
- Preserves backward compatibility
- Minimizes changes to the existing execution pipeline

3 Analysis through Reverse Engineering

This section presents the analysis of the SmartBugs framework through reverse engineering. The goal is to understand the architecture, identify key components, and evaluate how the proposed modification can be integrated.

3.1 Architectural Overview

SmartBugs follows a modular architecture composed of several Python modules, each responsible for a specific aspect of the system:

- **sb.cli**: handles command-line arguments and initializes execution
- **sb.settings**: manages configuration parameters
- **sb.smartbugs**: orchestrates the overall workflow
- **sb.tools**: loads and represents analysis tools
- **sb.tasks**: defines execution units (tasks)
- **sb.analysis**: coordinates execution of tasks
- **sb.docker**: executes tools inside Docker containers
- **sb.solidity**: handles Solidity compilation and pragma resolution

3.2 Package Diagram

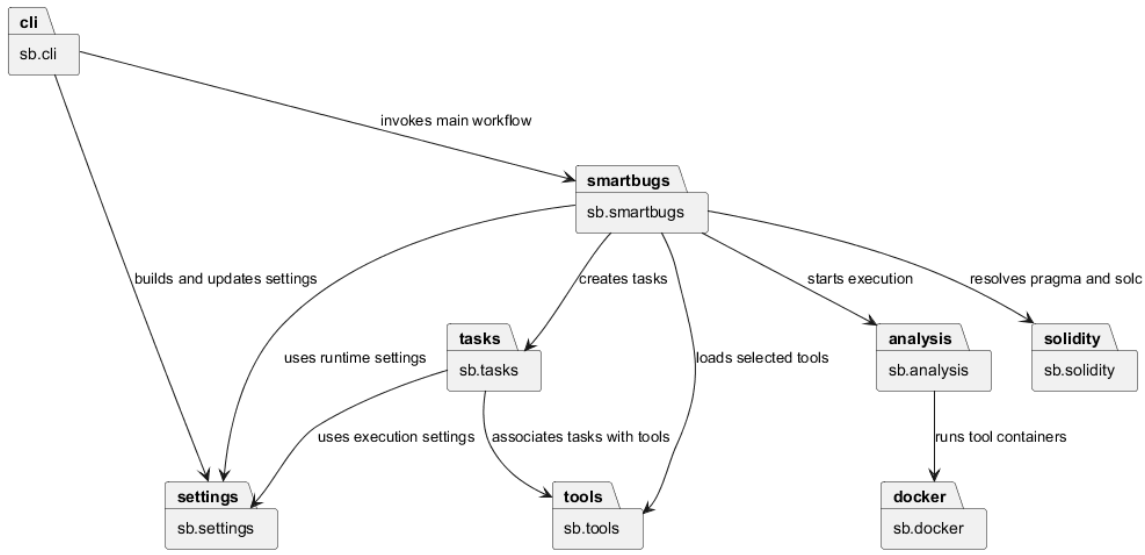

Figure 1: SmartBugs Package Diagram

The package diagram shows the main SmartBugs modules and their dependencies. sb.cli is the entry point of the system and is responsible for parsing command-line arguments and initializing the execution settings. sb.smartbugs acts as the main orchestrator, coordinating tool loading, pragma and solc resolution, task creation, and analysis execution. The execution phase is delegated to sb.analysis, which relies on sb.docker to run the selected tools inside containers. Auxiliary modules such as sb.io, sb.logging, and sb.errors have been omitted for readability, since they provide support services rather than core logic.

This architecture clearly separates:

- Configuration management
- Task generation
- Execution logic

This separation is crucial for enabling future extensions.

3.3 Dependency Analysis

To further analyze the relationships identified in the package diagram, we constructed a dependency matrix that quantifies the interactions between the core modules. Each dependency count is derived from explicit source-level interactions, including module imports, type-related imports, and concrete uses of imported modules or classes. String-based type annotations were not counted.

The dependency matrix was derived by analyzing the source code of the main modules and counting the dependencies between them. Each cell (i, j) represents the number of times module i depends on module j. In this study, the analysis focuses only on the core modules involved in the orchestration and execution workflow, while auxiliary modules such as `sb.io`, `sb.logging`, `sb.errors`, and `sb.cfg` were omitted for readability.

Table 1: Dependency Matrix of the Core SmartBugs Modules

Module	**cli**	**settings**	**smartbugs**	**tools**	**tasks**	**analysis**	**docker**	**solidity**
cli	0	3	2	0	0	0	0	0
settings	0	0	0	0	0	0	0	0
smartbugs	0	3	0	3	3	2	3	5
tools	0	0	0	0	0	0	0	0
tasks	0	2	0	2	0	0	0	0
analysis	0	2	0	0	4	0	2	0
docker	0	0	0	0	1	0	0	0
solidity	0	0	0	0	0	0	0	0

The dependency matrix highlights the structural relationships between the core modules of SmartBugs. Each cell represents the number of dependencies from a module (row) to another module (column).

The analysis clearly shows that `sb.smartbugs` is the central orchestrator of the system, as it exhibits dependencies toward almost all other core modules, including `settings`, `tools`, `tasks`, `analysis`, `docker`, and `solidity`. This confirms its role as the main coordination component responsible for managing the overall workflow.

The `sb.cli` module primarily depends on `settings` and `smartbugs`, reflecting its responsibility for parsing user input and delegating execution to the core system.

The `sb.analysis` module shows dependencies on `tasks` and `docker`, which is consistent with its role in executing analysis tasks through containerized tools.

Modules such as `settings`, `tools`, and `solidity` exhibit no outgoing dependencies within this subset, indicating that they act as supporting or utility components.

In particular, the strong dependency between `sb.smartbugs` and `sb.solidity` highlights the importance of Solidity-specific preprocessing in the analysis pipeline.

Overall, the matrix confirms a modular architecture with a centralized orchestration layer. This structure is particularly suitable for extension, as new functionalities, such as the integration of machine learning-based tool selection, can be introduced at the orchestration level without requiring significant modifications to the execution layer.

3.4 Class Diagram

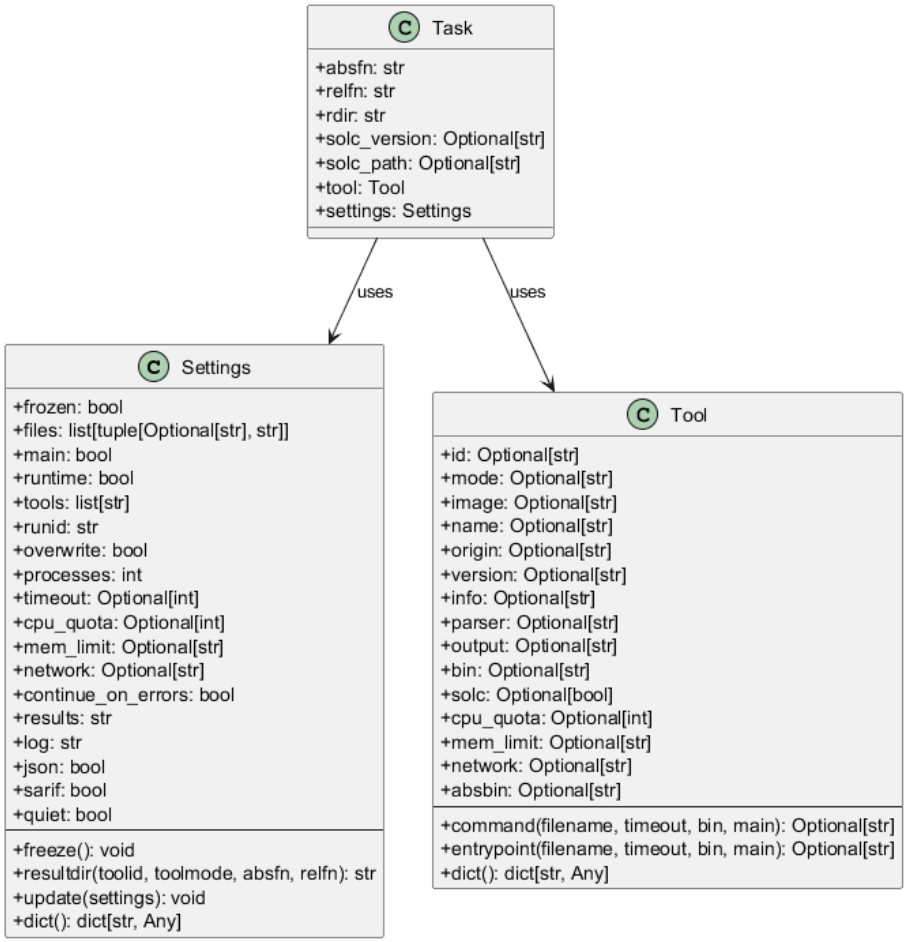

Figure 2: Core Class Diagram of SmartBugs

The class diagram focuses on the main data abstractions of SmartBugs that are directly involved in the configuration and execution of the analysis workflow.

Settings represents the global configuration of a SmartBugs execution. It stores all parameters required to control the analysis, including the list of input files, selected tools, execution constraints (e.g., timeout, CPU and memory limits), and output configuration. The class provides methods to update the configuration from external sources, normalize and freeze runtime parameters, and generate concrete result directory paths. This makes *Settings* the central coordination object shared across all phases of execution.

Tool models a single analysis tool supported by SmartBugs. It encapsulates both descriptive metadata (such as identifier, version, Docker image, and execution mode) and behavioral aspects, namely how the tool is invoked. The invocation logic is implemented through templated command and entrypoint definitions, which are instantiated at runtime depending on the specific task being executed. This abstraction allows SmartBugs to uniformly manage heterogeneous tools while keeping their execution configurable and extensible.

Task represents the fundamental unit of execution in the system. Each task corresponds to the application of a specific tool to a given input file. It aggregates all the information required for execution, including file paths, output directory, optional Solidity compiler configuration, the associated tool, and the global settings. In this way, *Task* acts as a bridge between configuration and execution, enabling the analysis engine to process multiple tool-file combinations in a structured manner.

The relationships between these classes reflect a clear separation of concerns. A *Task* depends on both *Tool* and *Settings*, combining them into an executable unit, while *Tool* and *Settings* remain independent abstractions. This design improves modularity and facilitates future extensions, such as introducing new tool selection strategies or modifying execution parameters, without altering the core execution model.

3.5 Sequence Diagram of the Current Workflow

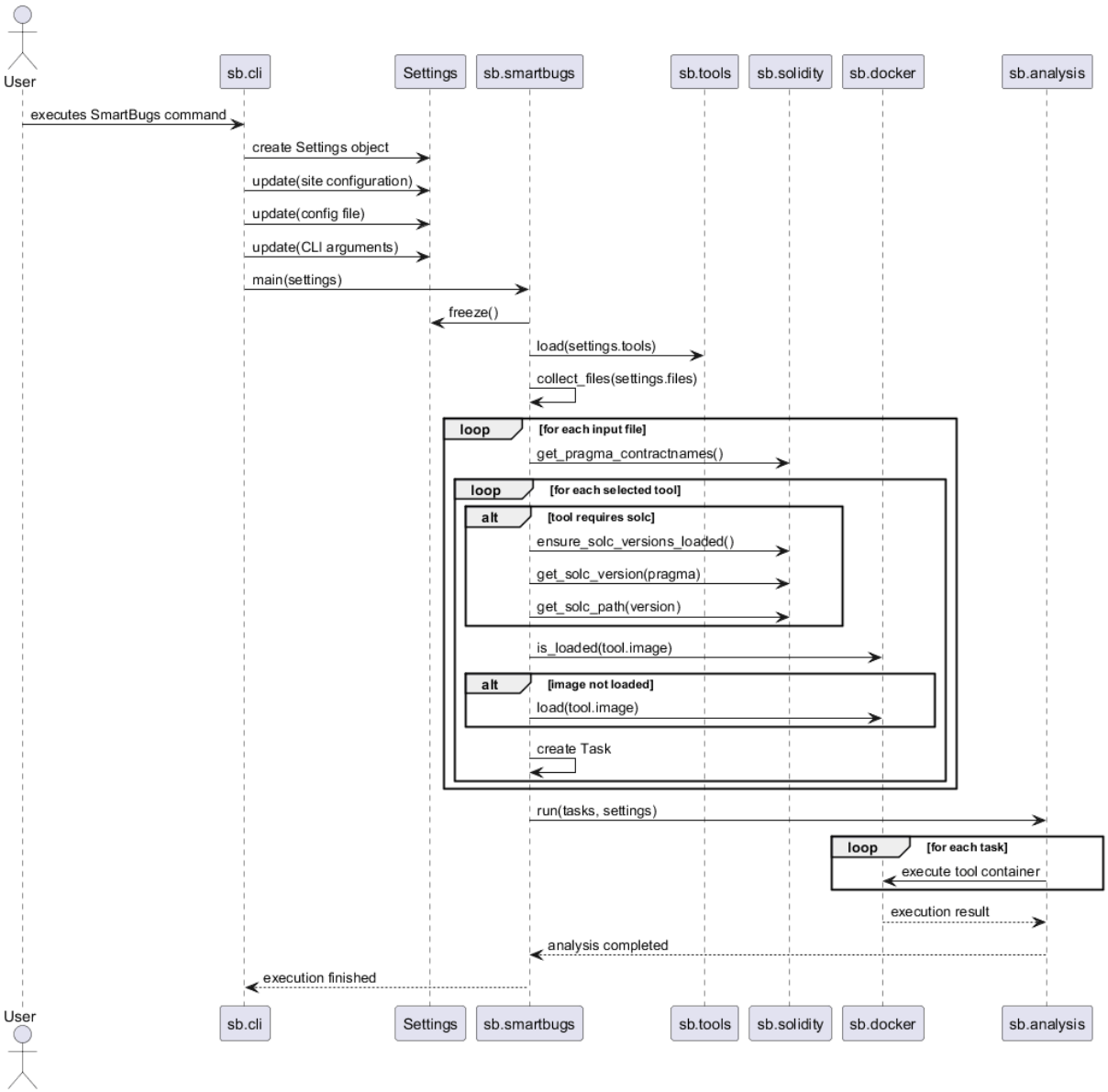

Figure 3: Current SmartBugs Execution Workflow

The sequence diagram illustrates the current execution workflow of SmartBugs. The process starts when the user invokes the framework through the command line interface. The `sb.cli` module creates a `Settings` object and progressively updates it using site-level configuration, optional configuration files, and command-line arguments.

Once the configuration is fully prepared, control is transferred to `sb.smartbugs`, which freezes the runtime settings, loads the selected tools, and collects the input files to be analyzed. For each input file, SmartBugs extracts Solidity-specific information, such as the pragma and contract names. Then, for each selected tool, it resolves the appropriate Solidity compiler when required, checks whether the corresponding Docker image is already available, and loads it if necessary. Finally, it creates the corresponding execution tasks.

After task generation, the execution phase begins. The list of tasks is passed to `sb.analysis`, which coordinates the execution of each task by invoking the appropriate Docker container through `sb.docker`. The results are then collected, and the overall execution terminates.

The diagram highlights a clear separation between configuration, task composition, and execution. This separation is important from an evolution perspective, since it suggests that modifications to tool selection can be introduced before the execution phase without necessarily affecting the existing task execution pipeline.

3.6 Analysis of External Components

Two external components were analyzed:

3.6.1 SCsVulLyzer

SCsVulLyzer is a feature extraction tool capable of analyzing Solidity smart contracts and producing a large set of features, including:

- AST-based features
- Opcode-based features
- Bytecode characteristics

Experimental validation confirmed that SCsVulLyzer can be executed correctly and produces consistent outputs.

3.6.2 Machine Learning Models

Eight pre-trained models were provided, each corresponding to a specific vulnerability category (SWC).

Through reverse engineering, the following observations were made:

- Models are implemented as **scikit-learn pipelines**
- Each model requires **41 input features**
- These features correspond to a subset of SCsVulLyzer outputs
- The output of each model is a **tool name** (e.g., `slither`, `mythril`, etc.)

Additionally:

- Models require **scikit-learn version 1.6.1** for compatibility
- Each model includes preprocessing steps (e.g., scaling, encoding)

3.7 Sequence Diagram of the Proposed Workflow

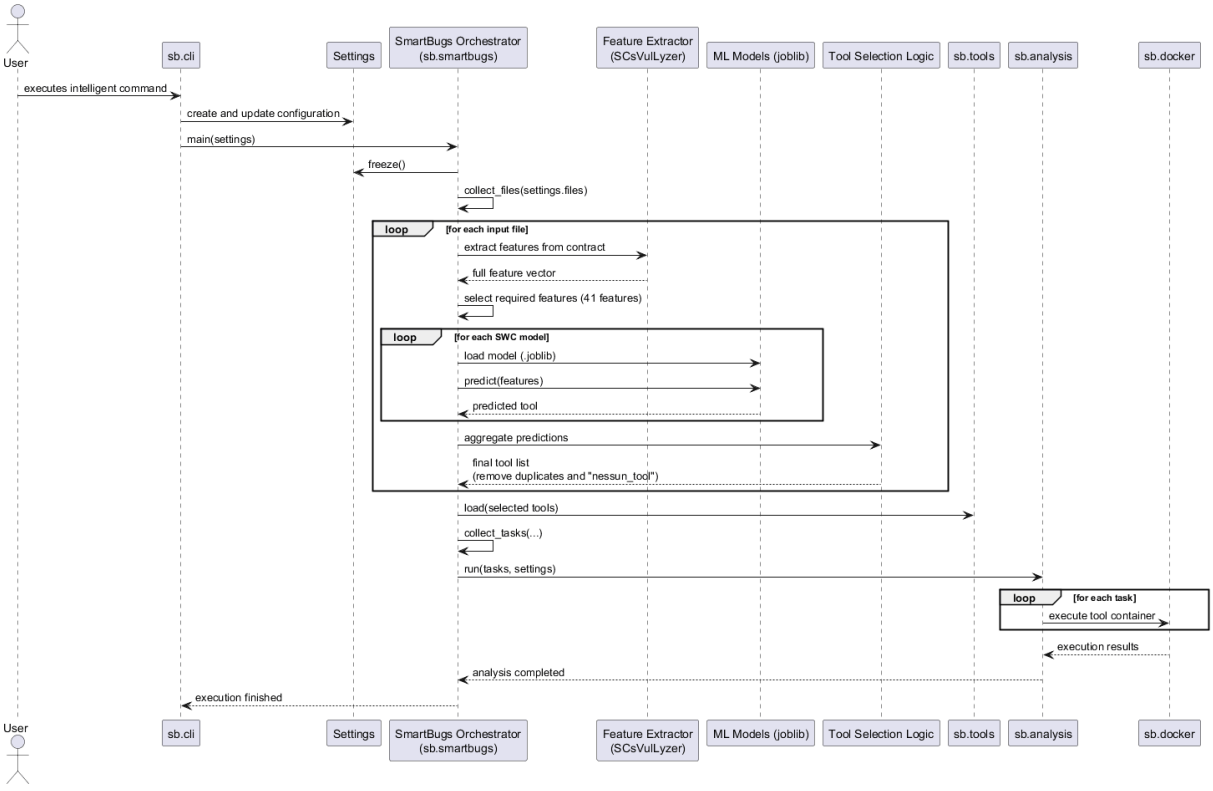

Figure 4: Proposed Intelligent SmartBugs Workflow

The proposed workflow extends the original SmartBugs execution pipeline by introducing an intelligent tool selection phase based on machine learning models. The process still starts from the command-line interface, where the configuration is created and finalized as in the original system.

After collecting the input files, the system introduces a new feature extraction step. For each smart contract, features are extracted using SCsVulLyzer. These features describe structural and behavioral aspects of the contract, including AST properties and opcode statistics.

From the extracted feature vector, only the subset of features required by the trained models is selected. This subset corresponds to the input expected by the machine learning pipelines.

The system then executes multiple pre-trained models, each associated with a specific vulnerability category (SWC). Each model receives the same feature vector and predicts the most suitable analysis tool for that category. This results in multiple tool predictions for a single contract.

The predicted tools are then aggregated and filtered. Duplicate tools are removed, and non-informative predictions (e.g., "nessun_tool") are discarded. The resulting set represents the tools that are expected to be most effective for analyzing the given contract.

Once the tool selection phase is completed, the workflow returns to the standard SmartBugs pipeline. The selected tools are loaded, tasks are generated, and the analysis is executed using Docker containers as in the original system.

This design preserves backward compatibility and maintains a clear separation between tool selection and execution. As a result, the existing execution infrastructure of SmartBugs remains unchanged, while the decision process for selecting tools becomes adaptive and data-driven.

3.8 Summary of Findings

The reverse engineering phase highlights that:

- SmartBugs is highly modular and suitable for extension
- The current limitation lies in manual tool selection
- External tools and models are compatible with the framework
- The integration can be achieved with limited architectural impact

4 Proposed Changes & Contributions

This section describes the proposed modifications introduced to extend the SmartBugs framework.

The goal of the project is to introduce an intelligent mechanism for automatic tool selection based on features extracted from smart contracts and predictions generated by machine learning models.

To improve modularity, maintainability, and impact analysis, the proposed solution is decomposed into three change requests (CRs), each addressing a specific aspect of the system.

4.1 CR_1: Integration of Feature Extraction Module

Description	Integration of a feature extraction module (SCsVulLyzer) to automatically extract relevant characteristics from smart contracts.
Motivation	SmartBugs currently operates without any feature-based analysis. Introducing feature extraction is necessary to enable intelligent decision-making for tool selection.
Priority	High [X] Medium [ ] Low [ ]
Effort	High [ ] Medium [X] Low [ ]
Consequences if not accepted	The system will not be able to support automated tool selection.
Dependencies	None

Table 2: CR_1: Feature Extraction Integration

4.1.1 Methodology

The feature extraction process is implemented by invoking an external tool capable of analyzing smart contracts and producing structured feature vectors. These features are then formatted and prepared as input for the machine learning models.

4.1.2 Expected Results

- automatic extraction of relevant contract features;
- standardized feature representation for further processing;
- minimal impact on the existing SmartBugs pipeline.

4.2 CR_2: Machine Learning-Based Tool Selection

Description	Introduction of machine learning models to predict the most appropriate analysis tools based on extracted features.
Motivation	Currently, tool selection is manual and requires domain expertise. Machine learning models enable automated and data-driven selection of tools.
Priority	High [X] Medium [ ] Low [ ]
Effort	High [ ] Medium [X] Low [ ]
Consequences if not accepted	Tool selection remains manual, reducing usability and potentially leading to suboptimal analysis results.
Dependencies	CR_1

Table 3: CR_2: Machine Learning-Based Tool Selection

4.2.1 Methodology

A set of pre-trained machine learning models is loaded from serialized files. Each model processes the extracted features and predicts the most suitable analysis tool. The predictions are then aggregated and filtered to remove invalid or duplicate entries.

4.2.2 Expected Results

- automatic prediction of relevant tools;
- improved accuracy in tool selection;
- reduction of manual configuration effort.

4.3 CR_3: Integration into SmartBugs Workflow

Description	Integration of the feature extraction and machine learning modules into the SmartBugs execution pipeline through a new intelligent command.
Motivation	The outputs of feature extraction and model inference must be connected to the existing SmartBugs workflow to enable automatic execution of selected tools.
Priority	High [X] Medium [ ] Low [ ]
Effort	High [X] Medium [ ] Low [ ]
Consequences if not accepted	The introduced components remain isolated and cannot be used within the SmartBugs pipeline.
Dependencies	CR_1, CR_2

Table 4: CR_3: Integration into SmartBugs Workflow

4.3.1 Methodology

The SmartBugs orchestrator is extended to support a new execution mode. When enabled, the system extracts features, runs the machine learning models, selects the appropriate tools, and executes them using the existing analysis pipeline.

The integration is designed to be modular and non-intrusive, ensuring that the original workflow remains unchanged when the intelligent mode is not used.

4.3.2 Expected Results

- seamless integration of intelligent tool selection into SmartBugs;
- preservation of backward compatibility;
- improved usability and automation of the analysis process.

5 Impact Analysis

To evaluate the effects of the proposed changes on the existing SmartBugs system, an impact analysis has been conducted for each change request.

The analysis is based on:

- reverse engineering of the source code;
- the package and class diagrams;
- the dependency matrix.

For each change request, we identify:

- **Starting Impact Set (SIS)**: modules directly affected by the change;
- **Candidate Impact Set (CIS)**: modules potentially affected through dependencies with SIS modules;
- **Actual Impact Set (AIS)**: modules effectively modified after implementation.

5.1 CR_1: Integration of Feature Extraction Module

The first change request introduces a new feature extraction component that operates independently from the core SmartBugs workflow.

Starting Impact Set (SIS)

The change mainly introduces a new module responsible for feature extraction:

- Feature Extraction Module (new component)

Candidate Impact Set (CIS)

Since the feature extraction is invoked before the standard SmartBugs pipeline and does not modify existing modules, its impact is minimal. No existing modules are directly affected.

Actual Impact Set (AIS)

- Feature Extraction Module (new)

Module	SIS	CIS	AIS
sb.features (new)	X	X	TBD

Table 5: Impact sets for CR_1

Discussion

The impact of CR_1 is low, as it introduces a new component without modifying existing functionality.

5.2 CR_2: Machine Learning-Based Tool Selection

The second change request introduces machine learning models for tool prediction.

Starting Impact Set (SIS)

- Model Inference Module (new component)

Candidate Impact Set (CIS)

The model inference depends on the output of the feature extraction module but does not interact directly with core SmartBugs modules.

Actual Impact Set (AIS)

- Model Inference Module (new)

Module	SIS	CIS	AIS
sb.models (new)	X	X	TBD
sb.features (new)		X	TBD

Table 6: Impact sets for CR_2

Discussion
CR_2 has limited impact on the existing system, as it introduces a new module that processes data without modifying the SmartBugs execution pipeline.

5.3 CR_3: Integration into SmartBugs Workflow

The third change request integrates the new components into the SmartBugs pipeline, affecting core modules.

Starting Impact Set (SIS)
Based on the change request and the proposed sequence diagram, the integration affects the following modules:

- **sb.smartbugs**: Orchestrator.
- **sb.cli**: The `sb.cli` module is directly impacted as it must be updated to pass new configuration parameters to the orchestrator.

Candidate Impact Set (CIS)
The Candidate Impact Set (CIS) is derived by identifying all modules that depend on, or are invoked by, the members of the SIS, based on dependency analysis. This set defines the scope for regression testing.

Propagation of impact: The modification of `sb.smartbugs` affects the generation of the tool list, which propagates along the execution pipeline. Modules such as `sb.tools` and `sb.tasks` are involved in validating and constructing execution tasks, while `sb.analysis` and `sb.docker` execute those tasks. Although these modules are not modified, they must be included in the CIS to verify that variations in tool selection do not introduce inconsistencies or runtime issues.

Therefore, the final Candidate Impact Set is defined as:

$$CIS = \{\text{sb.smartbugs, sb.tools, sb.tasks, sb.cli, sb.analysis, sb.docker, sb.features, sb.models}\}$$

The modules `sb.settings` and `sb.solidity` are excluded from the CIS as they act as independent foundational providers that do not consume the modified orchestration logic.

Actual Impact Set (AIS)
The Actual Impact Set (AIS) will be determined after the implementation of the proposed changes.

Module	SIS	CIS	AIS
sb.cli	X	X	TBD
sb.smartbugs	X	X	TBD
sb.tools		X	TBD
sb.tasks		X	TBD
sb.analysis		X	TBD
sb.docker		X	TBD
sb.features (new)		X	TBD
sb.models (new)		X	TBD

Table 7: Impact sets for CR_3

Discussion
CR_3 represents the most critical modification, as it integrates the new intelligent workflow into the existing system. However, the impact is controlled and limited to a small subset of core modules, ensuring maintainability and reducing the risk of introducing defects.

5.4 Overall Impact

The impact analysis shows that:

- CR_1 and CR_2 introduce new components with no impact on existing modules;
- CR_3 affects only a limited number of core modules;

- the majority of the SmartBugs system remains unchanged.

This confirms that the proposed solution is modular and non-intrusive, preserving the stability of the original system while extending its functionality.

6 Testing Strategy and Assessment

6.1 Testing Strategy

The testing strategy adopted for this project follows a layered approach, combining unit testing, component testing, and integration testing. The goal is to ensure both correctness of the new functionality and preservation of the existing SmartBugs behavior.

At the lowest level, unit tests validate isolated components such as feature selection, model loading, and prediction handling. These tests focus on verifying the correctness of individual functions without external dependencies.

At the component level, tests validate the interaction between closely related modules, such as the integration between feature extraction and model inference, or between tool selection logic and SmartBugs task generation.

At the system level, integration tests aim to validate the full workflow introduced by the intelligent command, including feature extraction, model inference, tool selection, and execution through the SmartBugs pipeline.

Mocking is used selectively to isolate external dependencies such as Docker execution and Solidity compilation, while still preserving realistic interactions between internal modules.

6.2 Adequacy Criteria

The adequacy criteria adopted in this project are primarily functional and change-oriented, rather than based on full structural coverage.

In particular:

- each newly introduced functionality must be covered by at least one dedicated test case;
- each modified module must be tested along the execution paths affected by the proposed changes;
- the main SmartBugs workflows must be re-executed to ensure that existing functionality remains unchanged;
- the intelligent command must be validated across all its phases, including feature extraction, feature selection, model inference, prediction aggregation, and tool execution;
- backward compatibility must be verified by testing scenarios where tools are manually specified by the user.

Given the nature of the project, adequacy is defined in terms of meaningful coverage of the impacted functionality rather than exhaustive code coverage.

6.3 Testing Tools

The existing SmartBugs project already adopts `pytest` as its primary testing framework. The test suite makes extensive use of:

- `pytest` for structuring and executing test cases;
- fixtures defined in `conftest.py` for reusable test data and environments;
- mocking utilities (e.g., `unittest.mock`) to isolate external dependencies such as Docker execution and Solidity compiler interactions;
- temporary filesystem utilities provided by pytest (e.g., `tmp_path`) for testing file-based workflows.

The same testing infrastructure will be reused for validating the proposed extension, ensuring consistency with the existing development practices.

6.4 Regression Testing Approach

The project adopts a selective regression testing strategy. Since the proposed modification introduces a new feature without altering the core SmartBugs execution pipeline, regression testing focuses on verifying that existing functionalities are not affected.

In particular, regression testing includes:

- re-execution of existing unit tests provided in the SmartBugs test suite;
- validation of the standard workflow with manually specified tools;
- verification of file collection, task generation, and analysis execution;
- comparison of outputs before and after the modification on representative input contracts.

This approach ensures that the new intelligent command integrates seamlessly without introducing regressions in the original system behavior.

6.5 Assessment of the Existing Test Suite

The SmartBugs project already includes a structured test suite based on `pytest`. The tests are primarily organized under a `tests/` directory and cover key components of the system.

In particular, the file `test_smartbugs.py` provides extensive coverage of the orchestration logic, including:

- file collection using various patterns and formats;
- task generation for different tool modes and configurations;
- handling of edge cases such as missing pragmas, duplicate files, and incompatible tools;
- validation of the main execution flow.

Additionally, `conftest.py` defines a rich set of fixtures that support testing through reusable contracts, mocked Docker clients, sample compilation outputs, and temporary file structures.

However, the current test suite is primarily focused on unit and component testing. Although some tests simulate multi-module interactions, they rely heavily on mocking and do not represent full end-to-end execution scenarios.

Furthermore, the `tests/integration` directory is currently empty, indicating the absence of a dedicated integration testing layer.

As a result, while the existing test suite provides a strong foundation for validating individual components and supporting regression testing, it is not sufficient to validate the proposed intelligent workflow in a fully realistic environment.

For this reason, additional integration tests will be introduced as part of the proposed changes. These tests will focus on validating the complete pipeline, from feature extraction to tool execution, ensuring correctness and robustness of the extended system.

Detailed test cases, execution reports, and regression testing results will be provided as separate testing documents, as required by the project guidelines.